

Ch4 Recommender Systems and Matrix Factorization

V1: Formulation

$$u_i \rightarrow I_1, I_2, I_3, I_4$$

User u_i

Item I_j

Recommends relevant items to a user based on historical data.

Mathematically,
A dataset can be represented using a matrix

	I_1	I_2	I_j	I_m
u_1				
u_2				
$\vdots$				
u_i				
$\vdots$				
u_n				

A_{ij} = Rating given by ~~user~~ u_i on I_j
(1 to 5)

OR

u_i watched I_j
(1 or 0)

If no rating, leave it blank.

Properties of Matrix A:-

- 1) Matrix A of ratings is very sparse.

Let $n = 1 \text{ Million}$ $m = 10k$

↓
users

10^6

↓
items

10^4

$n \times m = 10^{10}$ V large no = 10 Billo

Lets just say an avg. user rates 5 movies/items.

$5 \times 1M = 5M$ non empty cells.

Metric

Sparsity of A = $\frac{\# \text{ of non empty cells}}{\text{Total } \# \text{ of cells}}$

$$= \frac{5 \times 10^6}{10^{10}}$$

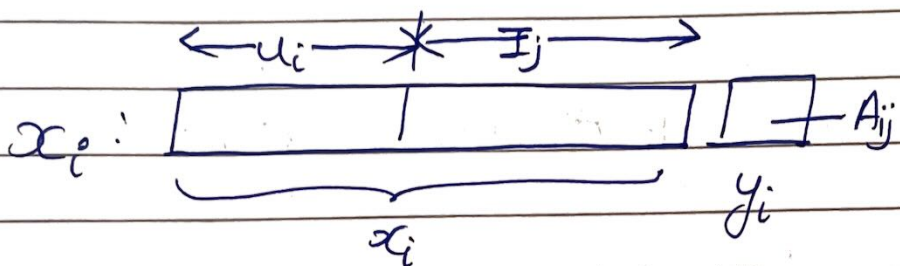
$$= 5 \times 10^{-4}$$

Task of Recommender System

$u_0 \div I_1, I_3, I_7, I_8$

↳ recommends

RS as a regression/classification
 ratings A_{ij} Boolean.



Challenge-

u_i - Feature of users

I_j - " " Items

Only A is data matrix given. nonempty cells

D_{train} has $A_{ij} \neq \text{NULL}$ (non

D_{test} - all the empty cells

How to come up with feature representation for u_i and I_j

Recommender System as a Matrix Completion Problem

$$A = \begin{bmatrix} & & \boxed{\checkmark} & \\ \boxed{\checkmark} & & & \boxed{\checkmark} \\ & & \boxed{\checkmark} & \\ & & & \end{bmatrix}$$

Some values of A_{ij} are given
Many A_{ij} are empty.

Challenge

Fill up the empty cells with reasonable values based on values in non empty cells using Regression/Classification.

V2: Content based VS Collaborative Filtering

2 types of Recommender Systems.

- 1) Content based
- 2) Collaborative Filtering.

2) Collaborative Filtering:

$u_1: M_1, M_2, M_3$
 $u_2: M_1, M_3, M_4$
 $u_3: M_1, ?$

M-Movie

matrix A

u_3 likes M_1 (liked by u_1 & u_2)
 Also, u_1 and u_2 both like M_3 .

Core idea - Users who agreed in the past tend to also agree in the future.

If this core idea is not assumed to be true, then we can't do collab filtering.

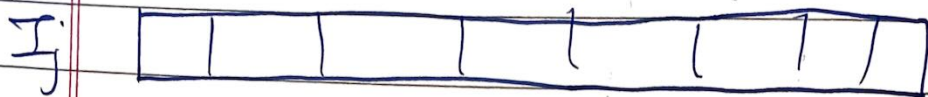
1) Content based:

Doesn't use the rating (A_{ij}) information. It represents I_j and u_i .

like m_j | I_j : Action, Drama, 90's, beach, actor, director.

u_i : likes action movies/RomCom

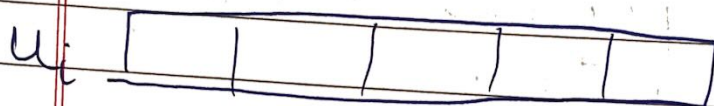
As soon as I have the above info, I can represent Item I_j as vector with many features.



f_1 = Action movie

f_2 = Decade of movie release

f_3 = Actors



f_1 = gender

f_2 = age group.

Do Classification/Regression on the combined vector which becomes as x_i and predict y_i .

$$x_i = I_j \times U_i$$

V3: Similarity based Algorithms

- 1) item-item (popularized by Amazon in 1998)
- 2) user-user

2) user-user similarity

	I_1	I_2	I_j	I_m
u_1				
u_2				
u_i				
u_n				

$$u_i = \begin{bmatrix} \text{Rating} & I_1 & I_2 & \dots & I_m \end{bmatrix}$$

u_i : user vector (sparse vector)
 BoW is also sparse vector.

$$\text{Similarity}(u_i, u_j) = \frac{\cosine(u_i, u_j)}{\|u_i\| \|u_j\|} = \frac{u_i \cdot u_j}{\|u_i\| \|u_j\|}$$

Similarity b/w u_i and $u_j = S_{ij}$

$S^u = S_{ij}^u$ = user similarity matrix

$S_{n \times n}^u$ matrix

Say, Task: recommend new items to user u_0 .

3) $u_0 \rightarrow u_1, u_2, u_7$ are the 3 most similar users to u_0 .

We are using rating data to compute similarity u_i & u_j .

4) Lets say pick items liked by u_1, u_2 & u_7 that are not yet watched by u_0 .

5) Recommend them to u_0 .

Problem: user's preferences change over time.

1) item-item based RS

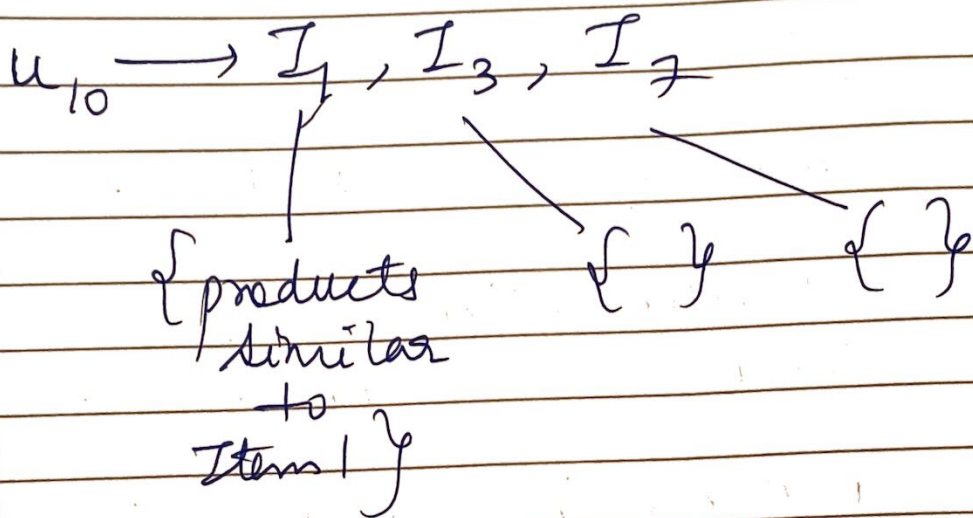
$$I_i = [r_{i1}, r_{i2}, \dots, r_{in}]$$

A =

	I_i	I_j

$$\text{Sim}(I_i, I_j) = \cosine(I_i, I_j)$$

Ratings on a given ~~to~~ item do not change significantly after the initial period.



Rule of Thumb : more users than items.
 eg - Amzn
 Netflix
 Youtube

Item ratings don't change much after the initial period.

Better to use item-item than user-user when there are more users in the system.

V4. Matrix Factorization: PCA, SVD

To solve RS problem use
Matrix Factorization/Decomposition

MF:

$A = B.C.D$ Matrix $A =$ Product
of B matrix, C matrix &
Matrix Decomposition, D matrix.

PCA (Principal Component Analysis)

Dimensionality Reduction
from high dimension to low dim.

PCA :- X - Data Matrix which
is centered

$$S_{d \times d} = \frac{X^T X}{n-1} - \text{Co-variance matrix}$$

normalizing factor

When I compute eigen values
of S say $\lambda_1, \lambda_2, \dots, \lambda_d$.

we get eigen vectors as $w_1, w_2, \dots$

To convert $d \rightarrow d'$ we take top eigen values and eigen vectors.

eigen vectors of S

ion.

$$S_{d \times d} = W_{d \times d} \Lambda_{d \times d} W_{d \times d}^T$$

Covariance matrix

eigen values of S

where,

$$W = \begin{bmatrix} \uparrow & \uparrow & \uparrow & \dots & \uparrow \\ w_1 & w_2 & w_3 & \dots & w_d \\ \downarrow & \downarrow & \downarrow & \dots & \downarrow \end{bmatrix}$$

eigen vector of S

eigen values of S

$$\Lambda_{d \times d} = \begin{bmatrix} \lambda_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \lambda_2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \lambda_3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \lambda_d \end{bmatrix}$$

diagonal matrix

w_d

This whole decomposition is often called as eigen decomposition or PCA is ~~decomposition~~ also called as eigen decomposition of S .

SVD - Singular Value Decomposition

It is a MF technique related to PCA.

Since PCA is performed on Co-variance matrix (square and symmetrical), but SVD could be performed on rectangular matrix. $X_{n \times d}$.

$$S = \text{Cov}(X)$$

~~$$X_{n \times d} = U_{n \times n} \Sigma_{n \times d} V_{d \times d}$$~~

~~$$X_{n \times d} = U \Sigma V^T$$~~

left sing
Cegenve.

where, U is $n \times n$

$$\Sigma = n \times d$$

$$V^T = d \times d$$

$$X_{n \times d} = U \Sigma V^T$$

Singular
Values S_i

Right
Singular
Vectors
of X

Singular value of X

$$\Sigma = \begin{bmatrix} s_1 & & & \\ & s_2 & & \\ & & s_3 & \\ & & & s_d \\ & & & & s \end{bmatrix}_{n \times d}$$

We know that

$$\frac{s_i^2}{n-1} = \lambda_i$$

So Σ and Λ are related
They form the central matrix
in both PCA & SVD.

$u_i = i^{\text{th}}$ column of U is i^{th}
eigen vector of $X \cdot X^T$
where X is $n \times d$ and
 X^T is $d \times n$.

upr vectors of X
chars of $X \cdot X^T$

$V_i = i^{\text{th}}$ column of V
 $= i^{\text{th}}$ eigen vector of
 $X^T X$ where X^T is
 $d \times n$ and X is $n \times d$.

(eigen vectors of $X^T \cdot X$)

classmate
Date _____
Page _____

VS: Matrix factorization: NMF or NNMF

NMF - Non negative Matrix factorization.

$$\text{eg} \rightarrow A_{n \times m} \rightarrow B_{n \times d} (C_{d \times m}^T)$$

where $B = n \times d$
 $C = m \times d$

$$\text{s.t. } B_{ij} \geq 0 \quad \forall i, j$$
$$C_{ij} \geq 0 \quad \forall i, j$$

Not positive but non negative
Since 0 is neither +ve nor -ve.

VB: Matrix factorization for Collaborative Filtering

$$A = \begin{bmatrix} \uparrow & \leftarrow I_j \rightarrow \\ u_i & \begin{bmatrix} \square \\ A_{ij} \end{bmatrix} \\ \downarrow & \end{bmatrix}$$

A_{ij} = Ratings on I_j by u_i
 it is a sparse matrix. As there are many empty cells.

$$\text{let } A = B * C^T$$

where, $A = n \times m$
 | |
 users items

$$B = n \times d \quad d > 0$$

$$d \leq \min(m, n)$$

$$C = m \times d$$

$$\text{then } A_{ij} = B_p * C_j$$

|
 ith
 row
 of B
 matrix

|
 jth row
 of C
 matrix

A_{ij} can also be written
as $B_p^T * C_j$

where $B_p = 1 \times d$ & $C_j = d \times 1$

Since $B_p =$ $\begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$ $n \times d$

$C =$ $\begin{bmatrix} \\ \\ \\ \end{bmatrix}$ $m \times d$

So $A_{ij} = B_p^T * C_j$

where $A_{ij} = 1 \times 1$.

A_{ij} has empty cells.

So we can do Optimization
problem

Find B and C,

$$\textcircled{1} \quad \text{optimization problem} \left[\begin{array}{l} \underset{B, C}{\operatorname{argmin}} \sum_{i,j \text{ valid}} (A_{ij} - B_i^T C_j)^2 \end{array} \right]$$

s.t. A_{ij} is not empty.

Use A_{ij} where there is valid data.

So this optimization should result in 0. This whole term is a squared loss.

$\textcircled{2}$ Find B and C

$\textcircled{3}$ matrix completion.

(Filling empty cells in matrix A)

$$\text{New matrix } \hat{A} = B * C^T$$

A = Actual Ratings

$\hat{A}$ = Predicted "

V7: Matrix Factorization for Feature Engineering

Let matrix A has user ratings

$$A = \begin{bmatrix} \uparrow & I_j \\ u_i & \square \\ \downarrow \end{bmatrix} \quad \begin{matrix} u_i = \boxed{} \\ d\text{-dim} \\ I_j = \boxed{} \\ d\text{-dim} \end{matrix}$$

Using the data in A_{ij}

$A_{n \times m}$ where $n = \text{users}$
 $m = \text{items}$

$$A = B \cdot C^T$$

$\begin{matrix} 1 \\ n \times d \end{matrix}$ $\begin{matrix} m \times d \text{ (not Transpose of } C) \end{matrix}$

$$B = \begin{matrix} & 1 & 2 & 3 & \dots & d \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ \vdots \\ \vdots \end{matrix} & \begin{bmatrix} \\ \\ \\ \\ \end{bmatrix} & \end{matrix}$$

$\begin{matrix} \text{users } n \\ u_i \\ n \times d \end{matrix}$

$$C = \begin{matrix} & 1 & 2 & \dots & d \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ \vdots \\ m \end{matrix} & \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \end{matrix}$$

$m = \text{items}$

$n = \text{users}$

$d = \text{dimensions}$.

$B_i = i^{\text{th}}$ row of user vector.

$$u_i \in \mathbb{R}^d$$

$C_i = i^{\text{th}}$ row of $C = I_i \in \mathbb{R}^d$

d should be greater than 0.

The d -dimensional representation using Matrix factorization has behaviour that u_i, u_j are very similar in their ratings of products/item.

So $\text{dist}(u_i, u_j)$ is also small

I_i and $I_j \in \mathbb{R}^d$ are also very similar based on ratings

V8: Clustering as MF

Kmeans problem can be given a dataset D having x_i in which you have to find centroids (k in no.) With set k in no.

$$\min_{C_i} \sum_{i=1}^k \sum_{x_j \in S_i} \|x_j - C_i\|^2, x \in \mathbb{R}^d \quad (1)$$

Let Z be a matrix s.t.

$$Z_{ij} = \begin{cases} 1 & \text{if } x_j \in S_i \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$Z = \begin{matrix} & \begin{matrix} 1 & 2 & \dots & j & \dots & n \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ k \end{matrix} & \begin{bmatrix} & & & & \\ & & & & \\ & & \boxed{Z_{ij}} & & \\ & & & & \end{bmatrix} \end{matrix} \quad k \times n$$

if $Z_{ij} = 1$

$x_j \in S_i$ formed by C_i

① changes ~~to~~ because of
 ②

$$\min_{C, Z} \sum_{i=1}^k \sum_{j=1}^n Z_{ij} \|x_j - c_i\|^2 \quad (3)$$

$$X = \begin{bmatrix} x_1 & x_2 & \dots & x_j & \dots & x_n \end{bmatrix} \quad (4)$$

~~③~~

$$C = \begin{bmatrix} c_1 & c_2 & c_i & \dots & c_k \end{bmatrix} \quad (5)$$

with (2), (4) and (5), (3) changes to:

$$\Rightarrow \min_{C, Z} \|X - CZ\|^2$$

where $X = C * Z$.

Kmeans = MF + Col-constraint + 0/1 constraint

V9: Hyperparameter Tuning

$$\text{Let } A_{n \times m} = \begin{matrix} & \begin{matrix} 1 & 2 & \dots & m \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ n \end{matrix} & \begin{bmatrix} & & & \\ & & & \\ & & \boxed{A_{ij}} & \\ & & & \end{bmatrix} \end{matrix}$$

$$A = B \cdot C^T$$

$n \times m \quad n \times d \quad m \times d$

where $d > 0$ and $d \leq \min(m, n)$

$$\downarrow$$

$$d < m \text{ \& } d < n$$

Hyperparameter = d .

How to determine d ?

1) Problem specific

Say, ratings $\Rightarrow u_i \rightarrow I_j = A_{ij}$

$u_i = 100 \text{ dim}; I_j \rightarrow 100 \text{ dim}$

2) Systematic way:

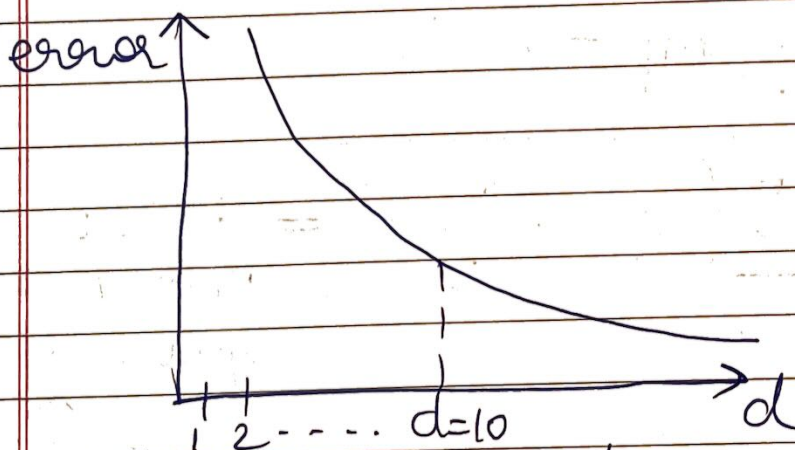
The optimization problem was:-

$$\min_{B, C} \sum_{i,j} (A_{ij} - B_i^T C_j)^2$$

Loss/error

$$error = \sum_{i,j} (A_{ij} - B_i^T C_j)^2$$

Plot:-



As $d \uparrow$, error $\downarrow$

As $d=10$ error falls very fast but as $d \uparrow$, error drop rate also false.
So, d here is Inflection point.

$d=10$ (inflection point) is a good tradeoff.

So, I try to pick smaller d where rate is high and d is also low.

$$\frac{\Delta \text{error}}{\Delta d} = \frac{\text{error}_i - \text{error}_{i+1}}{\text{error}_i - \text{error}_{i+1}}$$

V10: Matrix Factorization for recommen systems: Netflix Prize Solution

$$u_i, m_j \rightarrow \hat{r}_{ij}$$

$$\sqrt{\frac{1}{n} \sum_{i,j} (r_{ij} - \hat{r}_{ij})^2} = \text{RMSE}$$

Root mean Square error

↑
performance metric

A_{ij} $r_{u,i} \rightarrow$ rating u on i $q_i \rightarrow$ item vector $p_u \rightarrow$ user vector.1st optimization problem

$$\min_{q, p} \sum_{(u,i)} (r_{u,i} - q_i^T p_u)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

$(A_{ij} - B_i^T C_j)$ our notation

Squared loss

L_2 regularization

We should add our Regularization to optimization problem.

1) Solve this using SGD

Compute $\frac{\partial L}{\partial p_u}$; $\frac{\partial L}{\partial q_i} \rightarrow$ takes more time.

2) Alternating Least Squares (ALS)

a) Fix p_u as constants; Gradient

descent to find best q_i 's.

b) Fix q_i 's; Gradient descent to find good p_u 's.

loop steps a and b till you converge.

ALS is a faster algo.

Now,

Scalars $\left\{ \begin{array}{l} b_u = \text{bias (average rating across all users and items)} \\ b_i = \text{bias (average rating across all items)} \\ \mu = \text{mean (average rating across all users and items)} \end{array} \right.$

$$\min_{q, p, b} \sum_{u, i} \left(r_{u,i} - \mu - b_u - b_i - p_u^T q_i \right)^2$$

$$+ \lambda \left\{ \|q_i\|^2 + \|p_u\|^2 + b_u^2 + b_i^2 \right\}$$

$$\text{where } \mu = \frac{1}{n} \sum_{u, i} r_{u,i} \Rightarrow \mu$$

$n = \# \text{ ratings}$

VII: Cold Start Problem

If you have a new user in your ecosystem:-

- 1) new user
 - 2) new product/item
- } no rating data for both

	I_i
u_j	

empty row
column w/o
rating

1) For new user
we predict top items based on meta-data:-

- a) geo location
 - b) browser
 - c) device
- } metadata

(actual rating given by that user on that item)

So we recommend using metadata.

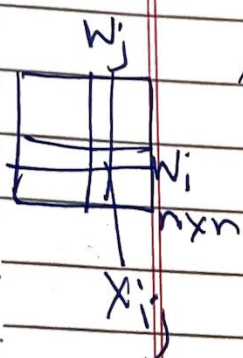
2) For new item

meta data

- a) category of product ~~des~~
- b) price
- c) description.

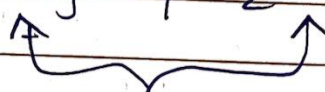
V12: Word vectors as MF

1) Co-occurrence matrix (X)



X_{ij} = # of times w_j occurs in the context of w_i .

review₁₀ = $w_1 w_j w_1 w_2 w_i w_1 w_3 w_4$



Neighbourhood of distance = '5'

n = no. of words.

2) $X_{n \times n} = U \Sigma V^T$ Perform SVD

$U = n \times n$

$\Sigma = n \times n$

$V = n \times n$

Truncated SVD $\rightarrow$ Discard some columns, keeping only k .

So, $n \times n$ becomes $k \times k$.

So $\hat{U}$ has $n \times k$
 $\hat{\Sigma}$ has $k \times k$
 $\hat{V}^T$ has $k \times n$

k is hyperparameter

Now multiply these matrices

$$\hat{X} = \hat{U} \hat{\Sigma} \hat{V}^T$$

3) $\hat{U}$ = rows of $\hat{U}$ as $u_1, u_2, \dots, u_n$.

$$u_i \in \mathbb{R}^k$$

word 1
word 2
word n.

Problem:-

Co-occurrence matrix is very large
 Soln $\rightarrow$ Instead of using all n -words
 use most m top words using
 tf-idf.

So instead of $n \times n$ Co-occurrence matrix use $n \times m$ " " matrix.

V13: Eigen Faces

For image data we used old technique called Eigen Faces for face recognition.

1) Flatten the photo pixels

$$A_{n \times d} \rightarrow \text{cov}(A) = S_{d \times d}$$

$$S_{d \times d} = W_{d \times d} \Lambda_{d \times d} W_{d \times d}^T$$

$$\Lambda = \begin{bmatrix} \lambda_1 & & 0 \\ & \lambda_2 & \\ 0 & & \lambda_3 & \dots \\ & & & \lambda_d \end{bmatrix}$$

$W_{d \times d}$ = eigen vectors

3) $d\text{-dim} \rightarrow k\text{-dim}$ $k < d$

↑
Dimensionality
Reduction

Use Top k - eigen values $\rightarrow$ eigen vectors

$$v_i \rightarrow v_o \in \mathbb{R}^d$$

$$4) A_{n \times d} W_{d \times k} = \tilde{A}_{n \times k}$$

V14 code

$$\|X\|^2 = \sum_i \sum_j X_{ij} \rightarrow \text{frobenius norm.}$$

M10

classmate

Date _____

Page _____

Agglomerative clustering

link in mail

task doing in smaticloud 3@gmail.com

- ↳ Read Agglo blog ✓
- ↳ Ask doubts on mail on how to use it for Donors choose dataset ✓
- ↳ Check Reply for Agglo Dendrogram doubt.
- ↳ Also check how to do HPT-Agglo.